



Task 2: Data Preprocessing for a Regression Problem

Follow-up to Task 1 (Exploratory Data Analysis) | Prepared by: Toqa Ghozlan

Objective

In Task 1 you explored the dataset: distributions, correlations, missing patterns, and outliers. In this task you will turn that raw dataset into a clean, model-ready dataset for a **regression** problem (predicting a continuous target, e.g., price, score, demand, cost). You will practice the full preprocessing pipeline that a real ML engineer applies before training any model, and you will justify every decision using what you found during EDA — preprocessing is not a fixed recipe, it depends on the data and the model you plan to use.

What You Will Deliver

- A cleaned, encoded, scaled dataset ready for modeling (train/test already split, no leakage).
- A short written justification for every major decision (why this imputation method, why this encoding, why these features were dropped, etc.).
- A reusable **scikit-learn Pipeline** that performs the full preprocessing automatically.
- A baseline regression model trained with proper cross-validation and at least one hyperparameter-tuned model.

1. Handling Missing Values

First quantify missingness (`df.isnull().mean()`) and check the **pattern**: is data missing completely at random (MCAR), at random (MAR), or not at random (MNAR)? The pattern should drive the method, not habit.

Technique	When to Use	sklearn / Tool
Drop rows	Missingness is very small (<2-3%) and random.	<code>df.dropna()</code>
Drop column	A column is missing >50-60% of its values and isn't critical.	<code>df.drop(columns=...)</code>
Mean / Median imputation	Numeric column, roughly symmetric (mean) or skewed / has outliers (median). Fast baseline.	<code>SimpleImputer(strategy='median')</code>
Mode / constant imputation	Categorical column, or you want a 'Missing' category that itself may be informative.	<code>SimpleImputer(strategy='most_frequent')</code>
KNN Imputation	Missingness relates to other features; small-to-medium datasets.	<code>KNNImputer</code>

Iterative Imputation (MICE)	MAR data with several correlated numeric features; you want statistically sound imputations.	IterativeImputer
Missing indicator flag	Add alongside any imputation when 'missingness itself' may carry predictive signal (MNAR).	MissingIndicator / add_indicator=True

Example: House Price Prediction (Ames Housing dataset): the 'LotFrontage' column is ~17% missing. Because frontage correlates strongly with neighborhood and lot area, KNNImputer or a group-wise median (median LotFrontage per Neighborhood) works far better than a single global median. Meanwhile 'PoolQC' is missing ~99.5% of the time because most houses simply have no pool — here missing is informative, so it is filled with the constant 'None', not imputed.

Example: Medical Insurance Cost dataset: 'bmi' is missing for a handful of patients who skipped that test. Since BMI correlates with age and smoking status, use IterativeImputer (MICE) conditioned on age, sex, and smoker rather than a flat mean — the flat mean would pull every imputed patient toward the population average and quietly weaken the smoker-BMI interaction you plan to engineer later.

Senior tip: always imputer-fit on train only, and always compare the imputed column's new distribution against the original non-missing distribution — if imputation flattens the variance (a common side-effect of mean imputation), your model will underweight that feature.

2. Detecting and Removing Duplicates

- **Exact duplicates:** identical rows across all columns — usually safe to drop. `df.duplicated().sum()` then `df.drop_duplicates()`.
- **Key-based duplicates:** same ID/entity appearing twice with different values (e.g., same house listed twice with updated price) — decide whether to keep the latest, average, or flag for manual review.
- **Near-duplicates:** rows that differ only by rounding or a data-entry mistake — worth a manual spot check before removing, since dropping too aggressively can quietly remove valid similar samples (e.g., two similar apartments in the same building).

Example: In a used-car price dataset, if 'car_id' repeats with identical mileage and price, it is a scraping duplicate and should be dropped. If it repeats with a different price and a later scrape date, keep the most recent record and drop the stale one.

Example: Student performance dataset: two rows share the same student name and scores but different 'student_id'. This looks like a duplicate but could be two different students with the same name — check a second key (birth date, class section) before dropping; deleting on name alone can silently remove a real, distinct student.

Senior tip: always check duplicates after preprocessing too — one-hot encoding or rounding during scaling can turn two originally-different rows into identical rows, which then leak into both train and test if you split after encoding.

3. Encoding Categorical Variables

Regression models need numeric input. The right encoding depends on cardinality, whether the category is ordinal, and which model family you will use (linear models are sensitive to encoding choice; tree-based models are more forgiving).

Technique	When to Use	Watch-out
One-Hot Encoding	Low-cardinality nominal features (no natural order), e.g., 'City', 'Fuel_Type'.	Can explode dimensionality if cardinality is high.
Ordinal Encoding	Category has a real order, e.g., 'Education Level' (High School < Bachelor < Master < PhD).	Never use on nominal data — it invents a false order.
Technique	When to Use	Watch-out

Target / Mean Encoding	High-cardinality nominal features, e.g., 'Zip Code', 'Neighborhood' (hundreds of categories) — encode by mean of target per category.	Must be computed only on the training fold to avoid target leakage.
Frequency Encoding	High-cardinality features where relative frequency itself is informative, e.g., 'Product Category'.	Loses semantic meaning of the category.
Binary / Hashing Encoding	Very high cardinality (thousands of categories) where one-hot is impractical, e.g., 'Store ID'.	Slight information loss from hash collisions.

Example: Bike-Sharing Demand dataset (regression: predict hourly rental count). 'Season' (4 values) → one-hot. 'Weather Situation' (ordinal: clear < misty < light rain < heavy rain) → ordinal encoding, since worse weather monotonically reduces demand. 'Station_ID' with 500+ stations → target or frequency encoding instead of one-hot.

Example: Real-estate dataset with 'Zip_Code' (300+ unique values): one-hot would create 300 sparse columns and hurt linear models. Target-encode it instead, but compute the mean price per zip code using K-Fold target encoding (encode each fold using stats from the other folds) — encoding on the full training set before splitting folds leaks target information and inflates validation scores.

Senior tip: for tree-based models, plain integer/label encoding of nominal features is often fine since trees split on thresholds, not linear weights — don't over-engineer one-hot encoding for LightGBM/XGBoost when it isn't needed.

4. Feature Scaling

Scaling matters for distance-based and gradient-based models; it is largely unnecessary for tree-based models (Random Forest, XGBoost, LightGBM), which split on thresholds regardless of scale.

Technique	When to Use	sklearn
StandardScaler (Z-score)	Features are roughly normal / no extreme outliers. Needed for Linear/Ridge/Lasso Regression, SVR, Neural Networks, PCA.	StandardScaler
MinMaxScaler	You need bounded values in a fixed range (e.g., [0,1]) — common before Neural Networks or when features must stay non-negative.	MinMaxScaler
RobustScaler	Data has significant outliers — uses median and IQR instead of mean/std.	RobustScaler
Log / Box-Cox / Yeo-Johnson	Target or feature is heavily right-skewed (e.g., price, income) — stabilizes variance and helps linear models.	np.log1p, PowerTransformer
No scaling needed	Tree-based ensembles (Random Forest, Gradient Boosting, XGBoost).	—

Example: Predicting house price: 'GrLivArea' ranges 300-5000 sq.ft while 'OverallQual' ranges 1-10. For Linear Regression or SVR, apply StandardScaler to both so gradient descent converges evenly and no feature dominates due to raw scale. If using Random Forest instead, scaling can be skipped entirely — it will not change the model's splits.

Example: Insurance charges dataset: the target 'charges' itself is heavily right-skewed (a few very expensive claims). Apply `np.log1p` to the target before training a Linear Regression, train on the log scale, then `np.expm1` the predictions back before computing MAE/RMSE in original dollars — evaluating only on the log scale can make the model look better than it really is in real-world units.

Senior tip: never fit the scaler on the full dataset before splitting, and never scale a one-hot encoded column — it's already 0/1 and scaling it just adds noise.

5. Feature Extraction (Creating New Features)

Technique	When to Use	Example
Date/time decomposition	Any datetime column.	Split 'Sale_Date' into Year, Month, DayOfWeek, IsWeekend, Season.
Domain ratios / interactions	Two raw features are more meaningful as a ratio or product.	'Price_per_SqFt' = Price / Area; 'BMI' = Weight / Height ² .
Polynomial features	Suspected non-linear relationship with a linear model.	PolynomialFeatures(degree=2) for Age vs. Insurance Cost.
Binning / discretization	A continuous variable has a threshold effect.	Bin 'Age' into life stages when the effect is non-monotonic.
Text feature extraction	A column contains free text.	TF-IDF on product 'Description' to get keyword importance features.
Dimensionality reduction (PCA)	Many correlated numeric features causing multicollinearity.	Compress 20 correlated sensor readings into 5 principal components.
Aggregation features	Data has a grouping key (customer, store, region).	'Avg_Order_Value_Per_Customer' aggregated from transaction history.

Example: Medical Cost dataset (predict insurance charges): create 'BMI_category' from BMI, and an interaction feature 'Smoker_x_BMI' — because the effect of BMI on cost is far larger for smokers than non-smokers, and a plain linear model cannot capture that interaction on its own.

Example: Bike-Sharing dataset: 'hour' (0-23) is cyclical — hour 23 and hour 0 are actually close in time, but a raw integer treats them as far apart. Encode it as $\text{hour_sin} = \sin(2\pi \cdot \text{hour}/24)$ and $\text{hour_cos} = \cos(2\pi \cdot \text{hour}/24)$ so the model sees the wrap-around correctly, instead of one-hot encoding 24 separate hour columns.

Senior tip: engineer features using only training-set statistics (e.g., group means), then apply the same transformation to the test set — recomputing aggregates on test data is a subtle but very common leakage source.

6. Feature Selection

Category	Technique	When to Use
Filter methods	Correlation threshold, Mutual Information, Chi-square (for encoded cats)	Fast first pass; remove features with near-zero correlation/relevance to the target, or drop one of two highly correlated features (multicollinearity).
Wrapper methods	Recursive Feature Elimination (RFE), Forward/Backward Selection	Smaller feature sets where you can afford to retrain the model repeatedly; more accurate than filter methods but slower.

Category	Technique	When to Use
Embedded methods	Lasso (L1) coefficients, tree-based feature_importances_, SHAP values	Selection happens as part of training; good default for medium/high-dimensional data.
Variance threshold	VarianceThreshold	Drop near-constant features that carry almost no information.

Example: Ames Housing has ~80 raw features. Start with a correlation matrix to drop near-duplicate features like 'GarageCars' vs 'GarageArea' (correlation > 0.9 — keep only one). Then fit a Lasso Regression; features whose coefficients shrink to zero are effectively removed. Cross-check with a Random Forest's feature_importances_ to confirm the same top predictors (OverallQual, GrLivArea, TotalBsmtSF) show up.

Example: Genomics-style wide dataset (e.g., 200 sensor readings, 500 rows): with far more features than samples, Lasso or RFE alone can overfit the selection step itself. Wrap feature selection inside the same cross-validation loop as model training (SelectFromModel inside a Pipeline) instead of selecting once on the full training set and then cross-validating the model — otherwise your CV score is optimistically biased.

Senior tip: feature selection is part of the model, not a one-time preprocessing step — if it 'sees' data outside its own training fold, your reported CV score will not generalize.

7. Outlier Detection & Treatment (Bonus but Important for Regression)

Regression targets (especially price/cost-like variables) are very sensitive to outliers, since MSE-based loss penalizes large errors quadratically.

- **IQR rule** ($Q1 - 1.5 \cdot IQR$, $Q3 + 1.5 \cdot IQR$): simple, good default for roughly unimodal data.
- **Z-score** ($|z| > 3$): assumes near-normal distribution.
- **Isolation Forest / Local Outlier Factor:** better for multivariate outliers that aren't extreme in any single column but are unusual in combination.
- Decide to **remove**, **cap/winsorize**, or **log-transform** — removing should be rare and only after confirming the point is a data error, not a rare-but-real case.

Example: A 10,000 sq.ft. mansion priced far above the rest of the market is a real, valid data point, not an error — capping it (winsorizing at the 99th percentile) is usually safer than deleting it, so the model doesn't silently lose the ability to predict high-end properties.

Example: Sensor/IoT dataset: a temperature reading of -999 clearly indicates a sensor malfunction, not a rare cold day. This is a data-entry/collection error and should be treated as missing (then imputed), not as a legitimate outlier to cap or keep.

Senior tip: always ask 'is this outlier a mistake or a rare truth?' before touching it — the two require opposite treatment (fix/remove vs. keep and let the model learn from it).

8. Train/Test Split and Avoiding Data Leakage

- Split the data **before** fitting any imputer, scaler, or encoder — fit only on the training set, then `.transform()` the test set.
- Use `train_test_split(test_size=0.2, random_state=42)` for a simple i.i.d. dataset.
- If the data is time-based (e.g., sales over months), split chronologically instead of randomly — never let future rows leak into training.
- Wrap the whole flow in a `Pipeline + ColumnTransformer` so every transformation is automatically refit only on training folds during cross-validation.

Example: Common leakage mistake: computing the global median of 'LotFrontage' on the full dataset (train+test) before splitting. This lets test-set information influence the training data. Correct approach: fit `SimpleImputer` only on `X_train`, then apply the same fitted imputer to `X_test`.

9. Cross-Validation

Technique	When to Use	sklearn
K-Fold CV (k=5 or 10)	Standard choice for i.i.d. tabular regression data.	KFold
Repeated K-Fold	Small dataset — reduce variance of the performance estimate by repeating K-Fold with different splits.	RepeatedKFold
Leave-One-Out (LOOCV)	Very small dataset (a few hundred rows or fewer).	LeaveOneOut
TimeSeriesSplit	Data has a time order (e.g., monthly demand) — never shuffle.	TimeSeriesSplit
GroupKFold	Multiple rows belong to the same entity (e.g., same patient/customer) and must stay together in one fold.	GroupKFold

Example: For a housing dataset, use 5-fold or 10-fold KFold cross-validation to get a stable estimate of RMSE/R2 instead of trusting a single train/test split, which can be optimistic or pessimistic just by chance depending on which rows land in the test set.

Example: Retail demand forecasting dataset (daily sales over 2 years): using plain KFold would let the model train on March 2024 and validate on January 2024 — training on the 'future' to predict the 'past'. Use TimeSeriesSplit instead so every validation fold only ever comes after its training fold in time.

Senior tip: always build every transform (imputer, encoder, scaler) inside the same Pipeline you cross-validate — if you preprocess once outside the CV loop, each fold's 'unseen' data was actually already seen during preprocessing.

10. Hyperparameter Tuning

Technique	When to Use	Tool
Grid Search	Small, well-understood hyperparameter space (e.g., 3x3 combinations) and enough compute to try all of them exhaustively.	GridSearchCV
Randomized Search	Large search space where trying every combination is too slow; sample a fixed number of random combinations instead.	RandomizedSearchCV
Bayesian Optimization	Expensive models (e.g., large gradient boosting models) where each trial is costly — use past trial results to choose the next smartly.	Optuna, scikit-optimize

Technique	When to Use	Tool
Halving Search	Many candidates but limited budget — eliminate weak candidates early using increasing data/resources.	HalvingGridSearchCV
Example: Tuning a Random Forest Regressor: use <code>RandomizedSearchCV</code> over <code>n_estimators</code>, <code>max_depth</code>, <code>min_samples_split</code>, and <code>max_features</code> with 5-fold CV, <code>scoring='neg_root_mean_squared_error'</code>, then refine the best region with a small <code>GridSearchCV</code> around the winning values.		
Example: Tuning an XGBoost Regressor on a mid-size dataset (~50k rows): a full grid over <code>learning_rate</code> x <code>max_depth</code> x <code>n_estimators</code> x <code>subsample</code> would take hours. Use <code>Optuna</code> with 50-100 trials and early stopping on a validation fold instead — it converges to a near-optimal region far faster than exhaustive grid search.		

Senior tip: always tune using the same cross-validation scheme you will report results with (e.g., the same `TimeSeriesSplit` folds), and keep a final untouched hold-out test set that is never used during tuning — tuning against your test set is the most common way beginners silently overfit their reported score.

11. Common Mistakes

Step	Common Mistake	Better Practice
Missing values	Imputing with the global mean/median before checking <i>why</i> data is missing, or imputing before the train/test split.	Diagnose the missingness pattern first (MCAR/MAR/MNAR); fit the imputer on train only and apply it to test.
Duplicates	Dropping duplicates using only a subset of columns without checking if rows are truly the same entity.	Verify with a secondary key before deleting; re-check for duplicates after encoding, not only before.
Encoding	One-hot encoding a high-cardinality column (100+ categories), or target-encoding on the full dataset before splitting.	Use target/frequency encoding for high cardinality, and always compute it with K-Fold or train-only statistics to avoid leakage.
Scaling	Fitting the scaler on the full dataset (train+test) before splitting, or scaling one-hot encoded (0/1) columns unnecessarily.	Fit scaler only on <code>X_train</code> inside a Pipeline; skip scaling for already-binary/tree-based workflows.
Feature engineering	Creating aggregated features (e.g., group means) using the full dataset, including test rows.	Compute aggregates on training data only, then map them onto the test set the same way a fitted transformer would.
Feature selection	Selecting features once on the full training set, then reporting cross-validated scores as if selection never saw those folds.	Perform selection inside each CV fold (e.g., <code>SelectFromModel</code> inside a Pipeline).
Outliers	Removing every statistical outlier automatically without checking whether it's a real rare case or a data error.	Investigate each outlier cluster; cap/transform real extreme values, remove only true data-entry errors.
Train/test split & CV	Using random <code>KFold</code> on time-ordered data, or letting rows from the same entity (patient, store) land in both train and test.	Use <code>TimeSeriesSplit</code> for temporal data and <code>GroupKFold</code> when rows share an entity.
Hyperparameter tuning	Tuning hyperparameters against the final test set, so the 'test' score is no longer an honest estimate of generalization.	Keep a locked hold-out test set; tune only using CV folds carved out of the training data.
General pipeline hygiene	Applying preprocessing steps manually and separately for train and test, risking small inconsistencies.	Wrap every step in a single sklearn Pipeline + <code>ColumnTransformer</code> so train and test are guaranteed to go through identical transformations.

Deliverables & Submission Checklist

- Jupyter Notebook with clearly labeled sections matching this task (missing values → duplicates → encoding → scaling → feature extraction → feature selection → outliers → split → CV → tuning).
- One sentence of written justification under each step explaining *why* that specific technique was chosen for this dataset.
- A single fitted `sklearn.pipeline.Pipeline` object saved with `joblib`, combining preprocessing + final model.
- A short results table: baseline model vs. tuned model, with MAE, RMSE, and R2 on the held-out test set.
- A brief 'What I would try next' paragraph (e.g., try target encoding instead of one-hot, try Gradient Boosting instead of Linear Regression).

Regression Evaluation Metrics — Quick Reference

Metric	What It Tells You
MAE (Mean Absolute Error)	Average absolute error in the same units as the target; robust to outliers, easy to explain to non-technical stakeholders.
MSE / RMSE	Penalizes large errors more heavily (squared); RMSE is in the same units as the target, useful when large mistakes are especially costly.
R-squared (R2)	Proportion of variance in the target explained by the model (1.0 = perfect, 0 = no better than predicting the mean).
Adjusted R2	R2 corrected for the number of features — use when comparing models with different numbers of features.

Extra Resources

Number	Resource
1	https://medium.com/womenintechology/data-preprocessing-steps-for-machine-learning-in-python-part-1-18009c6f1153
2	https://www.datacamp.com/blog/data-preprocessing
3	https://medium.com/@mehul.chourasia28/handling-outliers-a-comprehensive-guide-to-data-cleaning-3a0589c0b366
4	https://medium.com/@abhaysingh71711/outliers-in-data-science-a-comprehensive-guide-to-detection-and-treatment-0128a864cbb0
5	https://medium.com/data-science/how-to-detect-and-deal-with-multicollinearity-9e02b18695f1
6	https://www.ibm.com/think/topics/model-selection

